Report: An AI Agent for Generative Modeling

Source code: `https://github.com/yanruoz/gen-model-agent` BIOS 774, Project 2

1. System architecture

The agent takes a dataset folder and a free-text objective, and runs with one command: `./run.sh <dataset> "<objective>"`. `run.sh` prepares an isolated `venv/`, downloads the dataset's declared, checksummed file if it is missing, and starts Claude Code headless (`claude -p`). The agent's instructions (`AGENT.md`) are passed as an appended system prompt; the run loads no user or project settings, and a dedicated permission file lets the agent run only `venv/bin/python scripts/*`, read the repository, and write under `outputs/`. Anything else is denied without a prompt, so a run can never stall on a question.

The design separates *deciding* from *computing*. The language model reads the data profile and five skill files (objective interpretation, preprocessing, model selection, downstream use, reporting) and writes JSON decisions; every number and figure comes from deterministic, seeded Python scripts. Each stage reads and writes JSON under `outputs/<dataset>/`, so stages can be rerun or resumed independently:

1. **Profile** (`profiler.py`): size, modality (image or tabular), count-data flag, sparsity, class balance.
2. **Interpret the objective** into `objective.json`: task, success criterion, and the decision *proceed*, *proceed_adapted*, or *decline*. Checked by `validate_objective.py`.
3. **Plan** `plan.json`: held-out split, preprocessing, models, hyperparameters, seeds, each with a dataset-specific reason. For class balancing, a *headroom check* runs first (Section 2). Checked by `validate_plan.py`.
4. **Execute** (`run_plan.py`): preprocess once; for each model and seed, train, generate, and evaluate; then run the objective test the task calls for; then draw figures.
5. **Critique**: a fresh-context subagent (`gen-critic`) sees only the objective, plan, metrics, and training logs, never the builder's reasoning, and checks six things (below). The agent makes exactly one plan revision in response.
6. **Recommend** downstream uses in `recommendation.json`, checked by `validate_recommendation.py`.
7. **Report**: the agent writes `findings.md`; `report.py` builds the PDF from the JSON files.

Implemented models (PyTorch, small enough for a laptop GPU or CPU): an NB/Gaussian/Bernoulli VAE for tabular data; a RealNVP normalizing flow; for images, a conditional convolutional VAE, a conditional GAN (projection discriminator, hinge loss), and a conditional diffusion model (small UNet, classifier-free guidance, DDIM sampling).

2. How the agent makes generative-modeling decisions

Whether to model. `validate_objective.py` enforces the data checks in code: too few samples means decline; a task that needs labels on unlabeled data must be declined or adapted; near-balanced classes cannot be called a class imbalance problem; and a task with no implemented objective test on this modality (for example uncertainty quantification) must be declined or adapted, so the agent never trains models nobody can judge. For class balancing, a **headroom check** trains the same classifier (three seeds each) on real data, on real data with resampling, and on real data plus genuinely new real samples of the target class taken from rows the subsample left out. The last is the best case for any generator. If it does not beat resampling, no generator trained on this data can be expected to, and the agent plans accordingly.

Which model. Each family has a fixed capability profile (exact likelihood, latent space, conditional generation, reconstruction), and `validate_plan.py` rejects a model that lacks what the task needs (for example a GAN for anomaly detection, since it has no likelihood). The model-selection skill picks *one* primary model by objective and data (for example a negative-binomial VAE on raw counts for single-cell data, conditional diffusion for labeled images), and allows at most one comparator, only with a stated reason. This is deliberately not a benchmark.

Hyperparameters come from worked heuristics keyed to the profile (latent size by sample count, epochs as a maximum with early stopping on a validation slice of the training data, measured per-epoch timings to fit the budget). Every value needs its own reason; placeholders such as "default" are rejected by the validator.

Evaluation is fixed per modality, never chosen by the model, and always against held-out real data with a real-vs-real reference: C2ST, MMD, precision/recall, and distance to the closest training record (DCR) for copying; in a PCA space limited to components that generalize (tabular) or in the features of a classifier trained on real images (images, Fréchet distance and label agreement). Then the objective test: latent space vs a same-size PCA (representation), train-on-synthetic-test-on-real (replacing real data), the balancing test, AUROC on corrupted held-out samples vs non-generative baselines (anomaly detection), or error on masked held-out values vs mean and

kNN imputation.

Downstream use. Each recommended use must cite a metric; the validator checks the cited path and value against the metrics files and rejects privacy claims ("anonymized", "safe to share"), requiring a statement that no formal privacy guarantee is given whenever sharing comes up.

Critique. The critic checks whether any metric tests the success criterion, whether references are sane, whether seeds agree, whether training ran long enough, memorization and privacy-claim risk, and reasons contradicted by results.

3. Tools and AI models used

Claude Code (headless `claude -p`) is the only AI model in the loop; it plans and writes but never computes. Models and metrics use PyTorch 2.14, NumPy, SciPy, and scikit-learn; figures and reports use Matplotlib. Evaluation design follows standard practice (C2ST, MMD, kNN precision/recall, DCR, Fréchet distance, train-on-synthetic-test-on-real); the synthcity benchmark (Qian et al., NeurIPS 2023) is a reference point for tabular fidelity and privacy metrics. VAE sampling uses ex-post density estimation over the training latent codes (Ghosh et al., ICLR 2020).

4. Results

The numbers below come from rehearsal runs of the final pipeline, each a single `./run.sh` command on an empty `outputs/`. Generated reports 1 and 2 come from separate clean runs of the same commands. Because the agent's decisions can vary between runs, their numbers may differ slightly from the ones here.

PBMC 3k (2,700 cells × 32,738 genes, 97.4% zeros, raw counts, no labels). Objective: "Learn a low-dimensional representation and generate realistic synthetic cells for sharing." The agent read this as representation learning (primary) plus data generation. It kept the 2,000 most variable genes, held out 20% of cells, and chose a negative-binomial VAE on raw counts (latent size 10, two seeds) as the primary model, with a RealNVP flow on 20 principal components as the one comparator. The critic found that the flow overfit by epoch 13 and had one seed. The single revision lowered the flow's learning rate and size and added a second seed; the VAE runs were reused.

	VAE seed 0	VAE seed 1	flow seed 0	flow seed 1
C2ST AUC (real vs real reference ≈0.57)	0.865	0.842	0.855	0.809
MMD^2 (reference 0.0003)	0.0075	0.0053	0.0086	0.0054
precision / recall	0.94 / 0.74	0.94 / 0.80	0.58 / 0.98	0.58 / 0.98
DCR ratio (synthetic / held-out)	1.14	1.15	1.61	1.52
latent trustworthiness (PCA-10: 0.833)	0.803	0.794	–	–

The VAE matches simple statistics well (correlation of per-gene means 0.998, of zero fractions 0.993, median library size within 1%), but a classifier still tells its cells from real ones (C2ST 0.84–0.87), and its latent space keeps neighborhoods slightly worse than a same-size PCA. Synthetic cells sit farther from the training cells than held-out cells do (DCR ratio above 1), so there is no sign of copying. The flow has higher recall but low precision: it covers the real cells but also produces many off-manifold ones. The agent reported that both parts of its own success criterion failed, recommended neither the latent codes nor the synthetic cells as stand-ins for real cells, allowed the synthetic counts only as a marginal-level test fixture, and stated that no formal privacy guarantee is given.

PathMNIST (89,996 $28 \times 28 \times 3$ images, 9 tissue classes, imbalance 1.63×). Objective: "Generate realistic tissue images of each class that could replace the real data for training a tissue classifier." The agent chose a class-conditional diffusion model on a stratified 20,000-image subsample and tested replacement with train-on-synthetic-test-on-real (TSTR): the same CNN trained on 10,000 synthetic or 10,000 real images, three seeds each, scored on held-out real images. The first model (110 epochs) reached a utility ratio of only 0.66, and its validation loss was still improving at the last epoch. The critic flagged this, and the one revision retrained for 155 epochs.

	110 epochs	155 epochs (final)	reference
TSTR accuracy (real-trained: 0.913 ± 0.002)	–	0.763 ± 0.014	
utility ratio (synthetic / real accuracy)	0.66	0.84	1
Fréchet distance (classifier features)	26.5	21.9	0.29
C2ST AUC	0.982	0.972	0.506
label agreement (real classifier on generated images)	0.846	0.888	0.940

Longer training helped on every metric, but the synthetic images still cannot replace real ones: the classifier loses 15 accuracy points, and the weakest classes (6 and 7, label agreement 0.68 and 0.67) are the ones a downstream

user would most need. The agent recommended against replacement and allowed the images only as illustrative examples.

Robustness: different objectives, same data. On PBMC, "Recover gene expression values lost to dropout" was read as imputation. The agent trained only the VAE (the validator allows only VAEs to impute) and tested it by hiding 10% of each held-out cell's nonzero counts. The VAE's error (RMSE on $\log(1 + \text{count})$ 0.512) beat both the library-scaled mean (0.550) and kNN imputation (0.633), and the verdict was "conditional": useful before exploratory analysis, not proof that a zero was a dropout. "Flag unusual cells that may be doublets or low-quality captures" was read as anomaly detection: GANs are excluded (no likelihood), and the agent scored held-out cells against simulated doublets and gene-shuffled cells. It trained the NB-VAE (scored by its negative ELBO) and a PCA flow as comparator, two seeds each. On doublets the VAE reached AUROC 0.935 (both seeds), against 0.708 for a kNN-distance baseline; the flow (0.14) and PCA reconstruction error (0.11) scored doublets as *more* typical than real cells, a plausible reason (not tested) is that after normalization a doublet's profile is an average of two cells, which sits near the center of the data. Gene-shuffled cells were trivial for every method (AUROC 0.995–1.0), so the agent did not recommend the VAE for them over the simpler baselines. Its one positive verdict was "conditional": rank cells by the VAE score to shortlist candidate doublets for manual review, alongside standard count-based QC, since real doublets and low-quality cells were not tested.

During development, the class-balancing objective "Balance the rarest tissue class" on PathMNIST ran the headroom check: classifier accuracy was 0.912 on real data, 0.923 with resampling, and 0.923 with genuinely new real images of the rare class. Even perfect synthetic images could not beat resampling at a 1.6× imbalance. The check told the agent this before any generator was trained, and the result agreed: adding diffusion images gave 0.909.

Why no GAN ran. A conditional GAN (projection discriminator, hinge loss) is implemented and tested, but the agent never chose it. The model-selection rules pick one primary model per objective and allow one comparator with a reason. For labeled images the primary is conditional diffusion, which trains stably and conditions on class reliably; a GAN is the comparator only when the plan needs faster sampling and says why. A GAN has no likelihood or encoder, so the validator excludes it from anomaly detection, imputation, and representation learning, and on a few thousand tabular rows the skill treats it as the least stable choice. A GAN must also run with at least two seeds (about 8 minutes each here), because its results depend strongly on the seed. In the PathMNIST run the first diffusion model took 18.6 of the 30 minutes, leaving less than the 16 two GAN seeds need, and the retrain used the full budget. This is a choice the rules allow, not a missing capability, but it means the reports contain no direct GAN-vs-diffusion comparison, which is a gap: a GAN might sample sharper images, at the cost of mode coverage.

5. Strengths and limitations

Strengths.

- *The agent tests the user's goal, not just sample quality.* Each objective maps to a fixed test against held-out real data and a non-generative baseline (resampling, PCA, mean/kNN imputation, kNN distance), so a negative result is reported as one. In every run above, the agent's verdicts matched the numbers, including plain "not recommended" outcomes for the two deliverable objectives.
- *Decisions are checked in code.* Validators reject a model that lacks the needed capability, lazy reasons, budget overruns, uncited downstream claims, and privacy claims. The headroom check stops the agent from promising gains that even extra real data could not give.
- *The critique loop changed results.* On PathMNIST the critic's training-length finding raised the utility ratio from 0.66 to 0.84; on PBMC it fixed an overfit comparator.
- *Isolation and reproducibility.* One command; isolated venv; seeded training; runs load no user settings and can run only the project's scripts.

Limitations.

- *Small models and budgets.* Models are sized for a laptop and a 30-minute budget. Image fidelity is measured in the features of a small classifier trained on PathMNIST, not Inception FID, so the numbers are not comparable to published FIDs.
- *Few seeds and one split.* Most results use two or three seeds of one held-out split, so the reported spread covers training noise, not variation from a different split or mask.
- *Limited task coverage.* Uncertainty quantification has no objective test and is always declined or adapted; class balancing is tested only on images; categorical tabular features are not supported.
- *No privacy guarantee.* Distance-to-closest-record is evidence against copying, not differential privacy.
- *Run-to-run variation.* The model's decisions are not deterministic, and a critique-driven retrain can roughly

double the PathMNIST run time (about 65 minutes in the rehearsal).

- *Operational issue found in rehearsal.* One run lost write access after the agent ran `cd`, because path permissions were relative to the working directory. `run.sh` now passes absolute path rules and denies `cd`.